

Static Arrays, Dynamic Arrays & Strings

Static
cannot change size

$A = [1 \ 2 \ 3 \ 4 \ 5]$
0 1 2 3 4 → Indices

$\{A[4] = 7\}$

→ Mutable objects

→ Membership

$\{5 \text{ in } A: \text{False}\}$
True
O(n) operation

$A[2] = 3$ → constant time operation

$A[4] = 7$ shift →

Insertion

$A = [1 \ 2 \ 3 \ 4 \ 7]$
↑
5 while 2 3 4 7

keeping all the other stuff

losing 1 element

cause array is static

↳ takes O(n) time

↳ $A = [1 \ 5 \ 2 \ 3 \ 4]$ 7
← Shift

Deletion

Delete 5

takes O(n)

$A = [1 \ 2 \ 3 \ 4 \ X]$
0 1 2 3 4

Dynamic Arrays

BEST SERVICE

SECURITY

↳ can change size

↓ lists in python

appending elements →

Some is O(1)

Some is O(n)

under the hood

↳ only at end

Insertion/deletion

in middle or front of array is O(n)

Python keeps doubling the size of the static array under the hood every time you fill up the static array, it'll take O(n) time to shift everything to a new array otherwise all the others are going to take constant time O(1). most of the time we're doing O(1)

Operation	Array / List
Appending to end	* O(1)
Popping from end	O(1)
Insertion/deletion not from end	O(n)
Modifying an element, Random access	O(1)
Checking if element exists	O(n)

* appending at the end of the array take average O(1)



Strings → Immutable (in python)

"a b c d"

$O(n)$

S = "a b c d e"

0 1 2 3 4

S[1] = b → $O(1)$

Operation

Appending to end, popping from end,
Insertion/Deletion not from end, Modifying
an element, checking if element exists

Random Access

String (Immutable)

$O(n)$

$O(1)$

